

Midterm Exam

(12:35 PM – 1:50 PM : 75 Minutes)

- This exam will account for 25% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them in the spaces provided.
- There are 18 pages including 9 blank pages. Please use the blank pages if you need additional space for your answers.
- This is a *closed book, closed notes* exam. *No cheat sheets* are allowed.
- You are *not allowed to use calculators*.

Good Luck!

Question	Pages	Parts	Points	Score
1. Recurrences	2 – 4	(a) – (d)	$4 \times 4 = 16$	
2. Asymptotic Bounds	5 – 7	(a) – (d)	$4 \times 4 = 16$	
3. An Iterative Sorting Algorithm	8 – 9	–	8	
4. Recursive Insertion Sort	10 – 14	(a) – (b)	$7 + 8 = 15$	
5. Quicksort	15 – 18	(a) – (d)	$4 \times 5 = 20$	
Total	–	–	75	

NAME: _____

SBU ID: _____

Question 1. [16 Points] Recurrences. For each of the functions below either solve it using the Master Theorem (and state the case that applies) or state why the Master Theorem does not apply. Assume that all log's are of base 2 and $T(n) = \Theta(1)$ for $n \leq 1$.

(a) [4 Points] $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + (2 + \sin n) n \log^3 n$

(b) [4 Points] $T(n) + T\left(\frac{n}{4}\right) = 2 \cdot T\left(\frac{n}{2}\right) + n$

(c) [4 Points] $T(n) = 4 \cdot T\left(\frac{n}{2}\right) + n^{\sin n + \cos n}$

(d) [4 Points] $T(n) = 2^{100} \cdot T\left(\frac{n}{2}\right) + 2^n$

page intentionally left blank (use for your answers, if needed)

page intentionally left blank (use for your answers, if needed)

Question 2. [16 Points] Asymptotic Bounds. For each of the following statements, state whether it is *True* or *False*. Justify each answer. Assume that all log's are of base 2.

(a) [4 Points] $1 + \left(\left\lceil \frac{n}{2} \right\rceil - \left\lfloor \frac{n}{2} \right\rfloor \right) n = \Theta(n)$

(b) [4 Points] $n(\log n)^{100} = o(n^{1.000001})$

(c) [4 Points] $T(n) = \Theta(1)$, where $T(n) = \begin{cases} \Theta(n^2) & \text{if } n \leq 2^{100}, \\ n^{\frac{1}{\log n}} & \text{otherwise.} \end{cases}$

(d) [4 Points] $1 = \omega\left(\frac{1}{n}\right)$

page intentionally left blank (use for your answers, if needed)

page intentionally left blank (use for your answers, if needed)

Question 3. [8 Points] An Iterative Sorting Algorithm. Show that the ITERATIVE-SORT routine given below runs in $\mathcal{O}(n \log n)$ time on an input of size n , where n is a power of 2.

Here, a call to $\text{MERGE}(A, p, q, r)$ merges two sorted subarrays $A[p : q]$ and $A[q + 1 : r]$ into a single sorted sequence and stores that sequence in $A[p : r]$. This is the same MERGE routine that we saw in the class which is given in the appendix in case you want to take a look (though not required).

```

ITERATIVE-SORT(  $A[1 : n]$  )
     $\{n = 2^k \text{ for some integer } k \geq 0\}$ 

    1.  $m \leftarrow 1$ 
    2. while  $m < n$  do
    3.     for  $j \leftarrow 1$  to  $\frac{n}{2m}$  do
    4.          $p \leftarrow (j - 1) \cdot 2m + 1$ 
    5.          $q \leftarrow p + m - 1$ 
    6.          $r \leftarrow q + m$ 
    7.          $\text{MERGE}(A, p, q, r)$ 
    8.      $m \leftarrow 2m$ 
    9. return

```


page intentionally left blank (use for your answers, if needed)

REC-MERGE(A, B, n)

Input: Two non-overlapping sorted arrays A and B containing n numbers each, where n is a power of two.

Output: Merge A and B such that no number in A is larger than any number in B and both are sorted.

1. **if** $n = 1$ **then**
2. **if** the number in A is larger than the one in B **then** swap the two numbers
3. **else**
4. let A_L (resp. B_L) denote the left half of A (resp. B) and let A_R (resp. B_R) denote its right half
5. REC-MERGE($A_L, B_L, \frac{n}{2}$)
6. REC-MERGE($A_R, B_R, \frac{n}{2}$)
7. REC-MERGE($A_R, B_L, \frac{n}{2}$)
8. **return**

REC-INSERTION-SORT(A, n)

Input: An array A containing n numbers, where n is a power of two.

Output: The numbers in A rearranged in nondecreasing order of value.

1. **if** $n > 1$ **then**
2. let A_L denote the left half of A and let A_R denote its right half
3. REC-INSERTION-SORT($A_L, \frac{n}{2}$)
4. REC-INSERTION-SORT($A_R, \frac{n}{2}$)
5. REC-MERGE($A_L, A_R, \frac{n}{2}$)
6. **return**

Figure 1: The recursive version of the insertion sort algorithm.

Question 4. [15 Points] Recursive Insertion Sort. When Pramod¹ was a student, he designed a recursive version of the insertion sort algorithm with improved cache performance. Figure 1 shows the high-level structure of the algorithm. This question asks you to analyze its running time.

- (a) [7 Points] Let $T'(n)$ denote the worst-case running time of REC-MERGE while merging two arrays of size n each. What is the cost of the **divide** step of the algorithm? What is the cost of the **combine** step? Write a recurrence relation for $T'(n)$ and solve it.

¹Pramod Ganapathi – currently a faculty member of SBUCS.

page intentionally left blank (use for your answers, if needed)

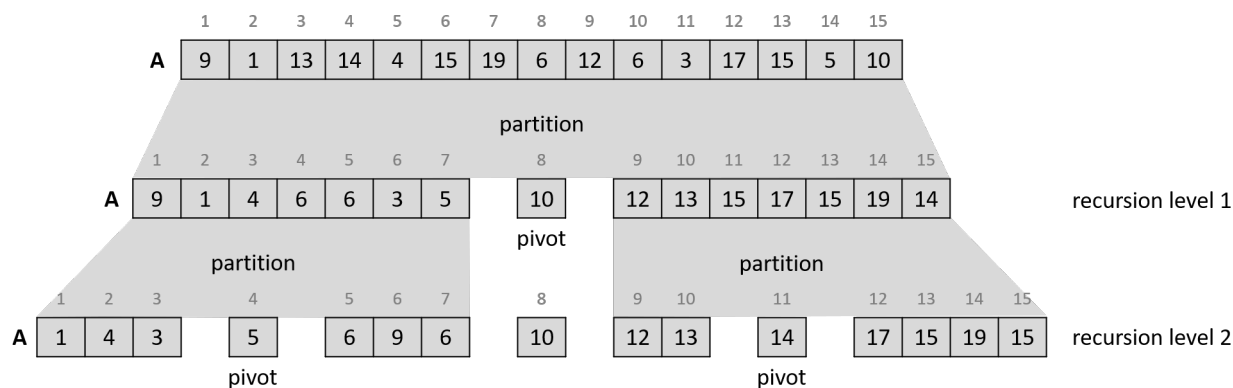
- (b) [**8 Points**] Let $T(n)$ be the worst-case running time of REC-INSERTION-SORT on an array of size n . What is the cost of the **divide** step of the algorithm? What is the cost of the **combine** step? Write a recurrence relation for $T(n)$ and solve it.

page intentionally left blank (use for your answers, if needed)

Question 5. [20 Points] Quicksort.

Recall the in-place partitioning algorithm we learned in the class which is used by quicksort to partition the numbers in the array or subarray it is working on around the pivot. We always chose the last element of the given array/subarray as the pivot.

Given an array $A[1 : 15]$ as input, the following example shows the state of A after partitioning in the first two levels of recursion of quicksort.



You can describe the state of A after partitioning in recursion level 1 as:

$[9, 1, 4, 6, 6, 3, 5] \ 10 \ [12, 13, 15, 17, 15, 19, 14]$

Then after partitioning in recursion level 2 as:

$[[1, 4, 3] \ 5 \ [6, 9, 6]] \ 10 \ [[12, 13] \ 14 \ [17, 15, 19, 15]]$

Now, given the following array $A[1 : 18]$ of numbers, you will have to describe its state after partitioning at recursion levels 1, 2, 3 and 4.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A	8	25	1	48	33	5	12	17	41	27	3	56	18	11	42	35	51	19

(a) [5 Points] Describe the state of A after partitioning at recursion level 1.

(b) [**5 Points**] Describe the state of A after partitioning at recursion level 2.

(c) [**5 Points**] Describe the state of A after partitioning at recursion level 3.

(d) [**5 Points**] Describe the state of A after partitioning at recursion level 4.

page intentionally left blank (use for your answers, if needed)

page intentionally left blank (use for your answers, if needed)

Appendix: The Merge Routine from Lecture 4

Merging Two Sorted Subarrays

Input: Two subarrays $A[p : q]$ and $A[q + 1 : r]$ in sorted order ($p \leq q < r$).

Output: A single sorted subarray $A[p : r]$ by merging the input subarrays.

```
MERGE ( A, p, q, r )  
  
1.   $n_1 = q - p + 1$   
2.   $n_2 = r - q$   
3.  Let  $L[1 : n_1 + 1]$  and  $R[1 : n_2 + 1]$  be new arrays  
4.  for  $i = 1$  to  $n_1$   
5.       $L[i] = A[p + i - 1]$   
6.  for  $j = 1$  to  $n_2$   
7.       $R[j] = A[q + j]$   
8.   $L[n_1 + 1] = \infty$   
9.   $R[n_2 + 1] = \infty$   
10.  $i = 1$   
11.  $j = 1$   
12. for  $k = p$  to  $r$   
13.     if  $L[i] \leq R[j]$   
14.          $A[k] = L[i]$   
15.          $i = i + 1$   
16.     else  $A[k] = R[j]$   
17.          $j = j + 1$ 
```